LAB 7 — Fuzzing for Input Validation Bugs

Bowling Alley Parser | Hypothesis Property-Based Testing

Student Name: Shreyas **SRN:** PES1UG25CS845

Application: bowling_alley_parser.py **Test file:** test_bowling_alley_parser.py

Objective

Use Hypothesis to generate unexpected and malformed inputs, identify input-validation bugs in the three parser functions, and harden the application so invalid inputs are handled without uncaught exceptions.

Functions tested

- **format_player_name(name)** — formats a player name for the scoreboard.
- **parse_frame(text)** — parses strike, spare, and two-roll frame notation.
- **parse_scoreboard(text)** — parses comma-separated name and score entries.

Fuzzing findings and fixes

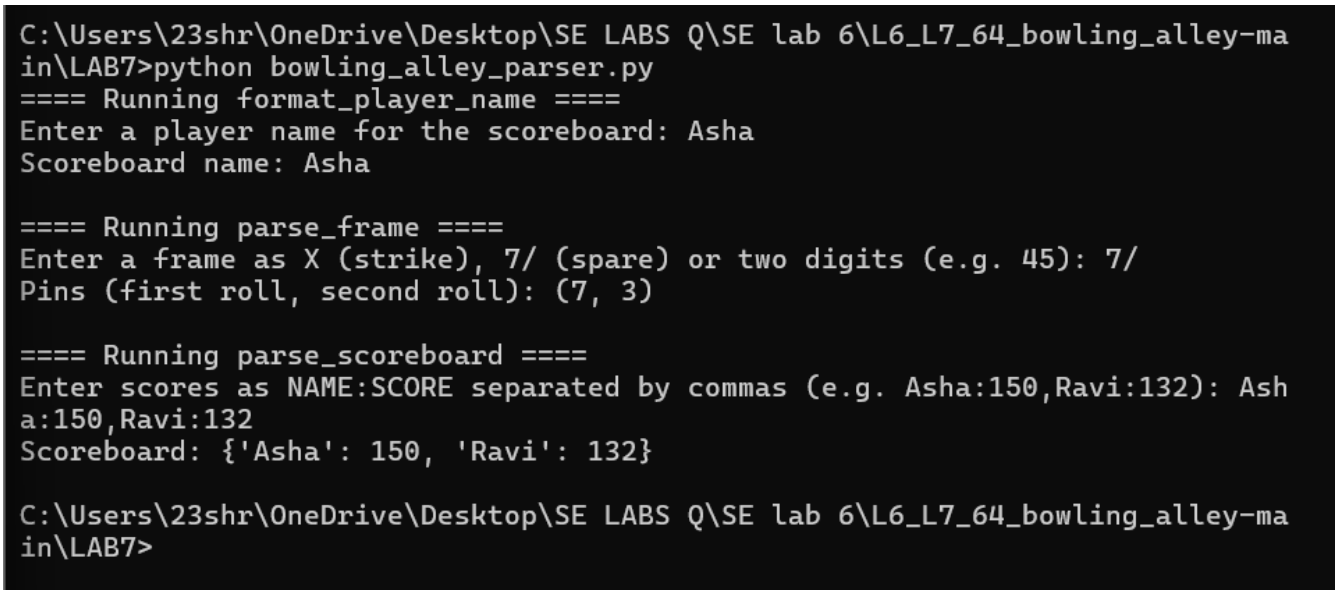
Hypothesis exposed failures for None values, empty strings, short frame strings, malformed frame characters, missing scoreboard separators, and empty or non-numeric scores. The fixed functions validate input types and structure before processing. Invalid player names return an empty string; invalid frames return (0, 0); invalid scoreboard entries are skipped, and a non-string scoreboard input returns an empty dictionary.

Test results

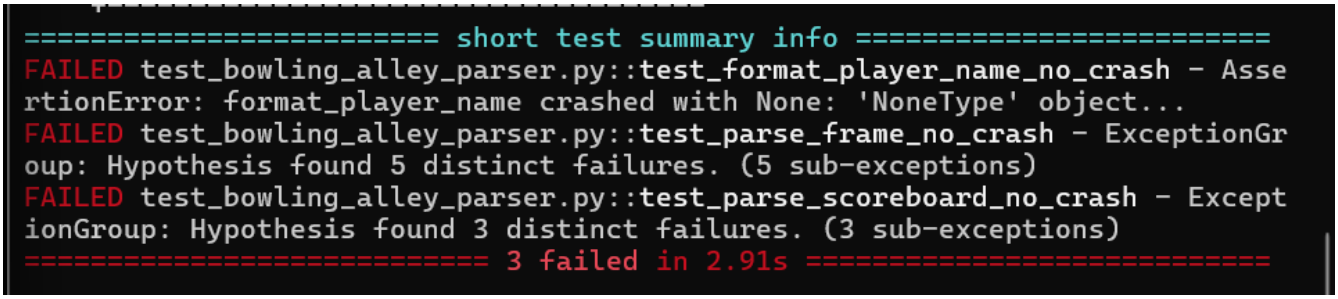
Before the fixes, all three property-based tests failed on generated edge cases. After the fixes, all three tests passed (3 passed). The application also retained the demonstrated normal-input behaviour: Asha is formatted as Asha, 7/ parses as (7, 3), and Asha:150,Ravi:132 parses into the expected score dictionary.

Screenshots

SS1 — Original buggy application output



SS2 — Hypothesis tests failing on the original application



SS3 — Fixed application output

```
C:\Users\23shr\OneDrive\Desktop\SE LABS Q\SE lab 6\L6_L7_64_bowling_alley-ma
in\LAB7>python bowling_alley_parser.py
==== Running format_player_name ====
Enter a player name for the scoreboard: Asha
Scoreboard name: Asha

==== Running parse_frame ====
Enter a frame as X (strike), 7/ (spare) or two digits (e.g. 45): 7/
Pins (first roll, second roll): (7, 3)

==== Running parse_scoreboard ====
Enter scores as NAME:SCORE separated by commas (e.g. Asha:150,Ravi:132): Ash
a:150,Ravi:132
Scoreboard: {'Asha': 150, 'Ravi': 132}
```

SS4 — Hypothesis tests passing after fixes

```
C:\Users\23shr\OneDrive\Desktop\SE LABS Q\SE lab 6\L6_L7_64_bowling_alley-ma
in\LAB7>pytest test_bowling_alley_parser.py
============================= test session starts =============================
platform win32 -- Python 3.12.3, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\23shr\OneDrive\Desktop\SE LABS Q\SE lab 6\L6_L7_64_bowling
_alley-main\LAB7
plugins: hypothesis-6.170.15
collected 3 items

test_bowling_alley_parser.py ...                                         [100%]

============================== 3 passed in 0.39s ==============================
```

Reflection

Note: The supplied Lab 7 handout describes the practical steps and deliverables, but the three reflection questions referenced in the LAB7 README are not printed in the provided handout. The following are suggested reflection responses based on the work completed; replace them if your instructor provided exact questions separately.

1. What is fuzz testing, and why was it useful in this lab?

Fuzz testing automatically supplies many unexpected and malformed inputs to a program. It was useful because Hypothesis found edge cases that the normal examples did not cover, including None, empty strings, incomplete frame notation, and malformed scoreboard entries.

2. What bugs did Hypothesis discover in the original application?

The original functions assumed their inputs were well formed. format_player_name crashed when given None. parse_frame crashed on None, empty or one-character strings, and non-numeric frame characters. parse_scoreboard crashed on None, missing or extra separators, and invalid or empty scores.

3. How did you improve the application, and how did you verify the fixes?

I added type and format checks before processing input. Invalid player-name input now returns an empty string, invalid frame input returns (0, 0), and invalid scoreboard entries are ignored. I reran the Hypothesis tests and all three passed. I also reran the application with valid sample inputs to confirm the expected outputs were preserved.